

TP 03 — Appels système POSIX en C

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	✓
CO2	Understand process scheduling, memory management, and file system organization algorithms	—
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	—

Contexte

Vous êtes développeur système junior dans une entreprise qui maintient des outils Linux. Votre responsable vous demande de comprendre et de déployer un utilitaire maison pour l'équipe : une commande `myls` qui liste un répertoire avec un format plus lisible que `ls` — type explicite de chaque entrée, tailles claires et un récapitulatif final. Le code source est fourni complet ; votre mission est de le **lire** pour le comprendre, de le **compiler**, de l'**installer** pour qu'il soit disponible partout, puis de **tracer** son exécution. La mission se déroule en deux objectifs, chacun piloté par un **script interactif** qui valide automatiquement votre travail et génère un rapport que vous montrez à l'enseignant.

Tableau de mission

Objectif	Mission	Script	Récompense
0	Préparer le poste de travail	— (PDF)	5 pts
1	De la commande au code source	<code>lab03_ex1.sh</code>	7 pts
2	Du code source à la commande installée (BOSS)	<code>lab03_ex2.sh</code>	8 pts

Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

- * Distinguer une commande interne d'une commande externe et expliquer le rôle du `PATH`
- * Lire un programme C et y reconnaître les en-têtes et les fonctions POSIX
- * Compiler un code source en exécutable avec `gcc`
- * Installer une commande dans `~/bin` et l'ajouter au `PATH`
- * Tracer une exécution avec `strace` et relier fonction de bibliothèque et appel système

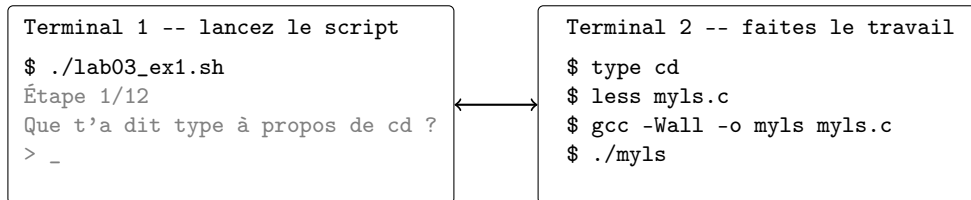
Pré-requis et matériel

- * Terminal Bash fonctionnel (WSL2 sur Windows, Terminal.app sur macOS, ou Linux natif)
- * Le compilateur `gcc` et le traceur `strace` installés
- * Avoir assisté au cours CM03 (appels système, API POSIX, `strace`) et réalisé les TP1 et TP2
- * L'archive `scripts_lab03.zip` téléchargée depuis le site du cours (voir Exercice 0)
- * Le fichier source `myls.c`, fourni complet dans l'archive (objectifs 1 et 2)

Déroulement et barème

Le TP se joue en **2 objectifs** de 30 minutes chacun. Pour chaque objectif, vous lancez le script correspondant (`./lab03_ex1.sh`, `./lab03_ex2.sh`) : il vous guide étape par étape, vérifie vos réponses, vous donne des indices en cas d'erreur, puis génère un **rapport** (`report_lab03_exN.txt`) que vous montrez à l'enseignant. Une discussion orale suit chaque objectif.

Pour chaque objectif, ouvrez **deux terminaux** dans le répertoire du TP : l'un pour lancer le script, l'autre pour effectuer le travail.



L'enseignant évaluera votre compréhension à travers des questions sur ce que vous avez fait, pourquoi ça marche, et le lien avec le cours vu en CM03.

Barème (sur 20) : Exercice 0 « Environnement prêt » (5 points) + deux objectifs notés sur 7 et 8 points (total 15). **Absence de machine opérationnelle = 0/20, quel que soit le reste.**

Annexe — Aide-mémoire des commandes du TP

Ce tableau ne remplace pas le cours. En cas de doute, utilisez `man commande` (section 2 : appel système, section 3 : fonction de bibliothèque) ou `commande --help`.

Repérage et documentation

Commande	Syntaxe	Rôle
<code>type</code>	<code>type commande</code>	Interne (builtin) ou externe (chemin vers un programme)
<code>which</code>	<code>which commande</code>	Localise une commande externe dans le <code>PATH</code>
<code>echo \$PATH</code>	<code>echo \$PATH</code>	Liste des dossiers où le shell cherche les commandes
<code>help</code>	<code>help commande</code>	Aide des commandes internes du shell
<code>man</code>	<code>man [2 3] nom</code>	Page de manuel (2 appel système, 3 bibliothèque)
<code>man -k</code>	<code>man -k mot-clé</code>	Recherche une commande ou fonction par mot-clé
<code>less</code>	<code>less fichier</code>	Consulte un fichier ; <code>/motif</code> cherche, <code>n/N</code> navigue

`man -k` s'appuie sur la base `whatis` : si elle ne renvoie rien, lancez `sudo mandb` (Linux) ou `sudo makewhatis` (macOS).

Lire, compiler et installer

Commande	Syntaxe	Rôle
<code>#include</code>	<code>#include <en-tête></code>	Importe les déclarations d'un en-tête (fonctions, types)
<code>wc</code>	<code>wc -l fichier</code>	Compte les lignes d'un fichier
<code>head</code>	<code>head -n N fichier</code>	Affiche les N premières lignes
<code>gcc</code>	<code>gcc -Wall -Wextra -o prog prog.c</code>	Compile le code source en exécutable
<code>file</code>	<code>file fichier</code>	Identifie le type réel du fichier (ex. ELF)
<code>./prog</code>	<code>./prog</code>	Exécute un programme du répertoire courant
<code>mkdir</code>	<code>mkdir -p chemin</code>	Crée un répertoire (+ parents manquants)
<code>cp</code>	<code>cp source cible</code>	Copie un fichier
<code>export PATH</code>	<code>export PATH="\$HOME/bin:\$PATH"</code>	Ajoute <code>~/bin</code> au <code>PATH</code>
<code>source</code>	<code>source ~/.bashrc</code>	Recharge la configuration du shell

La description d'un en-tête se trouve dans la page `man` des fonctions qu'il déclare : `<dirent.h>` via `man 3 opendir`, `<sys/stat.h>` via `man 2 stat`.

Traçage des appels système

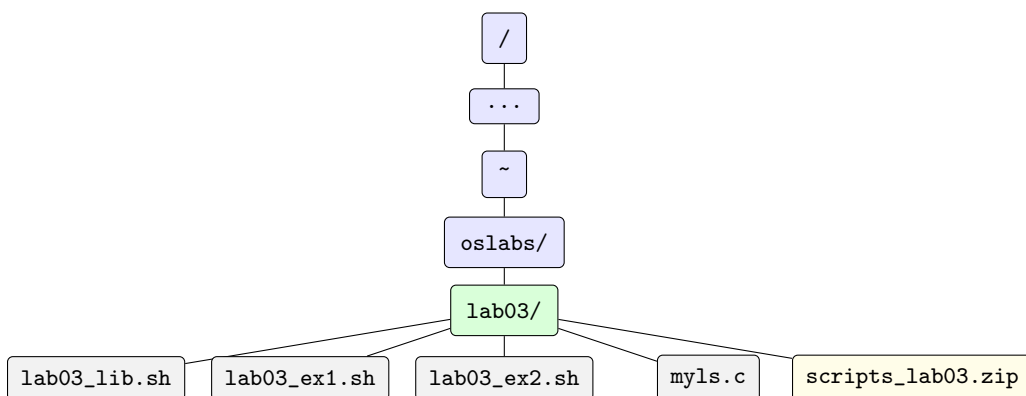
Commande	Syntaxe	Rôle
<code>strace</code>	<code>strace commande</code>	Affiche chaque appel système exécuté
<code>strace -o</code>	<code>strace -o f commande</code>	Enregistre la trace dans le fichier <code>f</code>
<code>strace -c</code>	<code>strace -c commande</code>	Résumé statistique (nombre d'appels par nom)
<code>strace -e</code>	<code>strace -e openat commande</code>	Ne garde qu'un appel (ex. <code>openat</code>)
<code>less trace.txt</code>	<code>less trace.txt</code>	Consulte et cherche dans la trace

1 Exercice 0 — Préparer le poste de travail

Durée : 15 min — 5 points « Environnement prêt »

Contexte

Avant toute mission, on prépare son poste. Ces commandes reviennent à **chaque TP** : à force de répétition, elles deviendront des réflexes. À la fin de cet exercice, votre arborescence doit ressembler à ceci :



Instructions

1. Aller à la racine : `cd /`
2. Revenir au répertoire personnel : `cd ~` ou `cd seul`
3. Entrer dans le dossier commun (Crée en TP2): `cd oslabs`
4. Créer le dossier du TP : `mkdir lab03`
5. Entrer dans le dossier du TP : `cd lab03`
6. Télécharger l'archive : `curl -O`
`https://liaoux.github.io/it-intro-to-os/static\_files/labs/lab03/scripts\_lab03.zip`
7. Extraire les fichiers : `unzip scripts_lab03.zip`
8. Rendre les scripts exécutables : `chmod +x *.sh`
9. Installer le compilateur et l'outil de trace : `sudo apt install gcc strace`
10. Vérifier les outils : `gcc --version` puis `strace --version`
11. Vérifier le contenu du dossier : `ls`

Attendu : vous trouvez les 5 fichiers du schéma — `lab03_lib.sh`, `lab03_ex1.sh`, `lab03_ex2.sh`, `mysls.c` et `scripts_lab03.zip`.

Si `gcc` ou `strace` manquent, installez-les avec `sudo apt update`
 Pour **WSL** `sudo apt install --reinstall build-essential`
 Pour **macOS** vous utilisez la commande `xcode-select --install`
 Puis `sudo apt install gcc strace`

2 Objectif 1 — De la commande au code source

Durée : 30 min — 7 points

Résumé de l'objectif

Ce premier objectif vous fait comprendre ce qu'est réellement une commande : la différence entre une commande interne et une commande externe, le rôle de la variable `PATH`, puis la lecture du code source `mys.c` pour y reconnaître ses en-têtes et ses fonctions POSIX. Le script `lab03_ex1.sh` vous guide ; vous cherchez dans le manuel (`help`, `man 2`, `man 3`) et dans le code, puis vous répondez à ses questions.

Lancez le script

Ouvrez deux terminaux dans le répertoire du TP : l'un pour lancer le script, l'autre pour effectuer le travail (voir l'illustration en début de sujet). Dans le premier, lancez :

```
./lab03_ex1.sh
```

Appelez le professeur avec votre `report_lab03_ex1.txt` en main.

3 Objectif 2 — Du code source à la commande installée

Durée : 30 min — 8 points

Résumé de l'objectif

Ce second objectif déroule le pipeline complet : comprendre la structure du programme, le compiler avec `gcc`, l'installer pour qu'il soit accessible partout, puis tracer son exécution avec `strace` pour révéler les appels système qui se cachent derrière les fonctions de bibliothèque. Le script `lab03_ex2.sh` vous guide à travers chaque étape.

Lancez le script

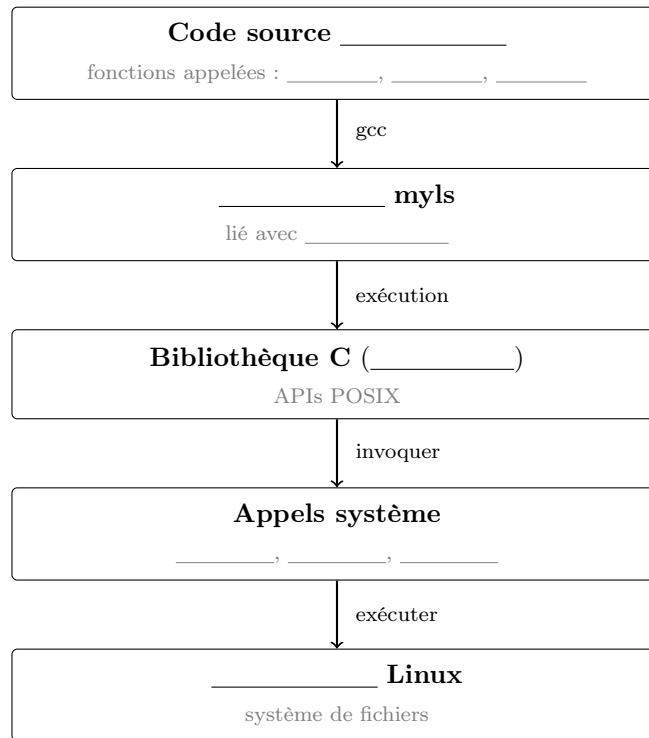
Ouvrez deux terminaux dans le répertoire du TP : l'un pour lancer le script, l'autre pour effectuer le travail (voir l'illustration en début de sujet). Dans le premier, lancez :

```
./lab03_ex2.sh
```

Appelez le professeur avec votre `report_lab03_ex2.txt` en main.

Exercice de synthèse — Complétez le schéma

Complétez les cases vides ci-dessous pour décrire le trajet d'un programme, du **code source** jusqu'au **noyau**. Aidez-vous de ce que vous avez fait dans les deux objectifs (lecture du code, compilation, installation, **strace**) et de l'annexe.



Note : **strace** montre uniquement les _____.